Task 1. Install Terraform and verify it

On Windows, install Terraform using PowerShell:

```
None
winget install Hashicorp.Terraform
terraform --version
```

If the command is not recognized, restart the terminal and check your PATH.

On Ubuntu Linux:

```
None
sudo apt-get update
sudo apt-get install -y wget gpg
```

Use HashiCorp's official installation instructions to add its repository and install the Terraform package. Then run:

```
None
terraform --version
```

Expected output: a Terraform version number.

Task 2. Create an EC2 instance in `us-east-1`

Create a folder named `terraform-ec2`, then create `main.tf`.

```
None
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

```
provider "aws" {
  region = "us-east-1"
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}

resource "aws_instance" "music_server" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  tags = {
    Name = "MusicStreamingServer"
  }
}
```

Explanation:

- `provider`: selects the AWS region.
- `data "aws_ami"`: finds a matching Amazon Linux 2 image.
- `aws_instance`: creates the EC2 virtual machine.
- `t2.micro`: the requested instance type; check current regional availability and your account's eligibility.

Task 3. Add your Owner tag

Add the `Owner` tag to the existing `tags` block:

```
None
tags = {
  Name  = "MusicStreamingServer"
  Owner = "Your Name"
}
```

Replace `Your Name` with your own name before applying. The tag appears in the AWS EC2 console.

Task 4. Initialize and create the instance

Configure AWS credentials using an AWS CLI profile or environment variables. Do not put access keys in `main.tf`.

```
None
aws configure
aws sts get-caller-identity

terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

Enter `yes` when Terraform asks for confirmation.

Verify the instance in AWS Console → EC2 → Instances, ensuring it reaches the `Running` state.

To clean up this lab afterward:

```
None
terraform destroy
```

This removes the Terraform-managed resources. Check the AWS console for any other resources or charges.

Task 5. Add a security group for SSH

Add this resource to `main.tf`:

```
None
data "aws_vpc" "default" {
  default = true
}

variable "my_ip_cidr" {
  description = "Your public IPv4 address in CIDR notation"
  type        = string
}

resource "aws_security_group" "ssh_access" {
  name        = "music-server-ssh"
  description = "Allow SSH from the specified client IP"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Update the EC2 resource with:

```
None
vpc_security_group_ids = [aws_security_group.ssh_access.id]
```

Create a `terraform.tfvars` file:

```
None
my_ip_cidr = "YOUR.PUBLIC.IP.ADDRESS/32"
```

Replace the placeholder with your actual public IPv4 address and `/32`. Do not use `0.0.0.0/0` for SSH.

Then run:

```
None
terraform fmt
terraform validate
terraform plan
terraform apply
```